JOUNIT: A PROOF-OF-WORK SETTLEMENT LAYER FOR HUMANS, MACHINES, ROBOTS AND AI AGENTS

DESIGN, SECURITY ANALYSIS AND IMPLEMENTATION REPORT OF THE JNT PROTOCOL *

Swamy Gadila
Jounit Protocol (community-owned, open source)
https://github.com/swamy18/Jounit

ABSTRACT

Jounit is an open-source cryptocurrency and settlement protocol designed around a single question that general-purpose blockchains do not answer well: how should a machine be paid for work that a third party can verify without re-running it? This paper describes the protocol end to end — its proof-of-work layer, its task marketplace, its wallet and agent-authorisation layer, and its security engineering — from the perspective of its sole author and implementer. The consensus core is a linear UTXO chain whose fork choice is heaviest cumulative work, whose proof of work is a three-stage hybrid (a SHA-256 chain, an integer-only neural network seeded from the epoch identifier, and a binding hash), and whose difficulty is adjusted by a multiplicative, windowed rule. Above it sits a Layer-2 task economy in which submitters escrow payment, workers claim and execute, anyone may challenge a result, and a stake-weighted committee of BLS-signing validators adjudicates disputes; a second Layer-2 mechanism moves value back to Layer 1 through an inclusion-proof-authorised, permissionlessly crankable exit. Above both sits the layer this paper's later sections are about: agent authority and credit as *consensus* state rather than as node-local bookkeeping — a grant published on chain that binds its owner to a named spender and a named agent, whose caps, allow-lists, window and nonce the chain enforces, and a loan whose lender signs the terms, whose principal really moves, and whose collateral answers for the debt. The paper documents the trust boundaries explicitly (what is trusted, what is not, and why reputation is deliberately excluded from consensus), the concrete security mechanisms that were built and the audit-driven improvements that followed them, the wallet and key-management design, the operational tooling (HTTP API, keyless MCP surface, worker SDK, command line), the open-source and governance posture, and the limitations that remain before mainnet deployment.

Keywords Proof of Work · Agent Payments · Escrow · Task Markets · Autonomous Agents · Consensus Security · Post-Quantum Signatures

1 Introduction

1.1 The problem

Every general-purpose blockchain was designed for a world in which the party holding a key is a person, a company, or a device operated by one. That assumption breaks in two directions when software begins to act on a principal's behalf. First, an autonomous agent that receives a payment must be able to prove what it did and to be paid for it without a human approving every step. Second, a principal that delegates spending authority to such an agent needs a way to bound that authority — by amount, by recipient, by time window — and to revoke it, without re-authorising each transaction by hand.

* *Citation*: **Gadila, S. Jounit: A Proof-of-Work Settlement Layer for Humans, Machines, Robots and AI Agents. Preprint, 2026. DOI:000000/11111.**

The second failure mode is the more consequential one, and it is not specific to cryptocurrencies. Agent frameworks are proliferating rapidly [1, 2, 3, 4], and robotic platforms are standardising on shared middleware such as the Robot Operating System [5]. Both create economic actors — software that consumes resources on somebody's behalf — while the payment rails remain institution-centric. A language-model agent that rents compute, a warehouse robot that consumes electricity, and a research pipeline that buys data are all in the same position: they can produce a receipt, but they cannot enforce one.

Payment is only the visible part of the problem. The deeper issue is *verifiability*. If a machine claims it produced a result, the payer has three unattractive options: trust the claim, re-run the work, or hire an arbitrator they both accept. Re-running is often impossible (the work may have consumed a one-shot resource), and a shared arbitrator is a trusted party — the exact thing a blockchain exists to avoid.

1.2 The claim

Jounit (ticker `JNT`) is an attempt to build the missing settlement primitive directly into a cryptocurrency: a chain that can act as escrow and arbiter for work performed by machines. The economic claim is deliberately narrow and testable:

> If a machine can produce a result that anyone else can check without re-running it, then that machine can be paid for that result without either side trusting the other.

Everything in the protocol is arranged to make that sentence true in practice: the escrow holds the payer's money before the work starts, the verifier is a deterministic function of committed inputs, the adjudicator is a quorum of stake-weighted validators rather than a named party, and the payout path is ordinary on-chain value movement rather than a bespoke accounting entry.

1.3 Contributions

This paper is a design-and-implementation report rather than a theoretical result. Its contributions are:

- **A concrete hybrid proof of work.** A three-stage puzzle that keeps the security of a SHA-256 chain while attaching a bounded, epoch-seeded neural stage, with a registry-enforced work-share band and integer-only arithmetic throughout [6, 7].
- **A two-layer architecture.** A conventional UTXO Layer 1 for durable settlement, and a Layer-2 task economy that is replayed deterministically from Layer-1 blocks rather than run by a separate consensus [8, 9].
- **A trust-minimal bridge.** Layer-2 deposits and exits are authorised by on-chain inclusion proofs rather than by a privileged operator, and the exit crank is permissionless [10].
- **An agent-first authorisation layer, enforced by the chain.** A grant published on chain binds its owner to a named spender and a named agent, and the chain enforces the per-transaction cap, the tumbling per-window cap, the allow-lists, the validity window and the nonce; a delegated grant that widens its parent is refused, and revocation reaches every descendant. The same applies to credit: the lender signs, the principal moves, the collateral answers for the debt [2].
- **A second implementation of those rules.** The authority and credit rules are written twice — once in the protocol, once in an independent reference derived from the specification — and the two are compared scenario by scenario, so a rule that refuses for a different reason is a defect rather than a matter of taste.
- **A documented security programme.** An explicit threat model, a concrete list of implemented mechanisms, an audit-driven remediation history, a fuzzing campaign, and an honest statement of the gaps that remain.
- **An open governance posture.** Permissive dual licensing, community ownership of the name, separation of the trademark from the licence, and a stated refusal of post-launch governance in the style of Bitcoin [11, 12].

1.4 Scope and honesty statement

This paper describes the protocol as implemented in the reference repository [13, 14]. The chain is *pre-launch*. The repository's own readiness record states plainly that no independent third-party security review has been completed, and this paper does not claim otherwise [15]. Where a feature is experimental, that is said in the text. Numbers quoted as measurements — test counts, fuzzing executions, benchmarks — are reported as they appear in the repository's verification record [16, 17], because the project has a deliberate policy of replacing fabricated measurements with either a measurement or an explicit "not measured".

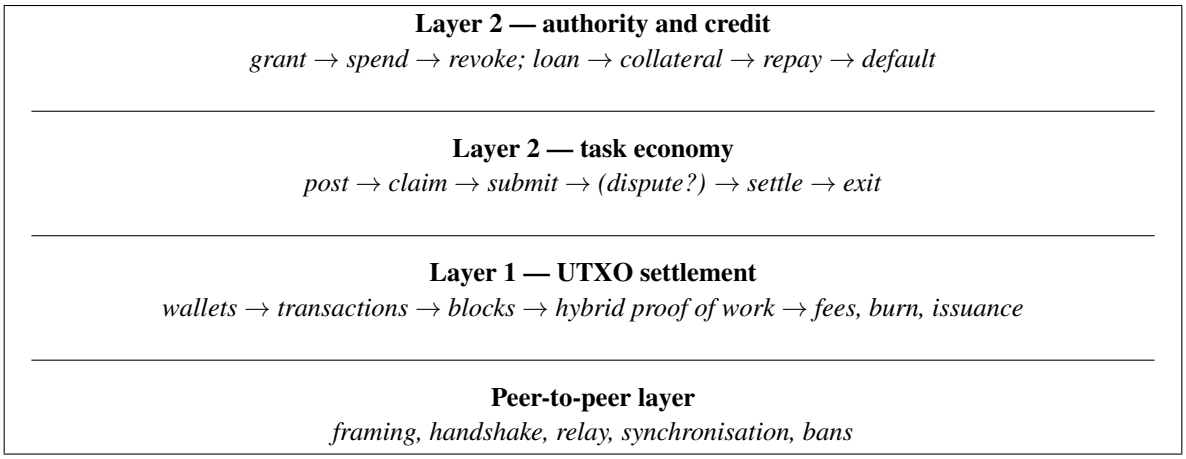


Figure 1: The three-layer structure of Jounit. Layer-2 transactions are carried inside Layer-1 blocks and committed by the block's Merkle root; Layer-2 state is rebuilt by deterministic replay, so no second consensus protocol exists.

2 Jounit at a glance

Jounit is a cryptocurrency with two layers that have different jobs.

Layer 1 is a conventional UTXO chain: wallets, transfers, blocks, a peer-to-peer network, a fee market and a bounded issuance schedule. Nothing about Layer 1 is specific to artificial intelligence. It exists to be a durable settlement layer, and it deliberately inherits the well-understood properties of Bitcoin [11, 18] rather than inventing a new ledger model.

Layer 2 is the part that makes the compute work worth doing. It is a task economy. A *submitter* posts an escrow for a described computation; a *worker* claims it, executes it and submits a result hash; anyone may *challenge* that result; a committee then decides whether the work was correct, and settlement moves the money accordingly.

Layer 2 also carries the authority and the credit. A *grant* is the owner's signed permission for a named agent to spend the owner's money within stated limits; a *position* is a loan one agent made to another. Both are consensus state, committed by the block's Layer-2 state root, so every node enforces the same limits rather than each node deciding for itself. This is the part of the design that makes the paper's central claim usable rather than merely stated: a fleet of agents can be given bounded spending authority that the network itself checks, and one agent can lend to another with a signature, a collateral and a deadline that every node can see.

2.1 Design principles

Five principles recur throughout the codebase, and they explain most of the otherwise surprising choices.

One code path, three profiles. The development specification requires that the fast test profile be *the same code* with different parameters, never a second implementation. Every consensus constant therefore lives in one parameters module, and the `mainnet`, `testnet` and `mini` profiles differ only in numbers. The practical consequence is that anything tested on `mini` is the production code path, and that a consensus bug cannot hide in a profile-specific branch.

Solving is hard, verifying is easy. This is the property that makes a blockchain a blockchain [11]. Jounit keeps the property and changes the work: the proof of work is not pure hashing, so that hardware useful for AI workloads is not left idle, but the security-critical part of the puzzle remains a plain SHA-256 chain.

No floating point, anywhere. The consensus core computes in integers end to end. Weights derive from the epoch identifier by SHA-256, forward passes accumulate in `i64` and wrap to `i32`, and every arithmetic operation that could overflow is refused rather than wrapped. A wrapped sum would compare small and pass a bound — that is a security property, not a style preference.

Fail closed. Ambiguous cases refuse rather than guess. An empty recipient list in a spending policy authorises nothing (not "anything"); a node with no configured credit issuer refuses every credit mint; a checkpoint presented to a node with no committee fails rather than verifying against nothing.

Structural, not documentary. Where a guarantee matters, it is expressed in the type system or in the shape of a data structure rather than in a comment that a future maintainer could forget. The canonical reference that decides a fraud dispute is *computed by the arbitration engine* from the task's committed fields and cannot be passed in as an argument. The credit module exposes no function that accepts a reputation score at all, so the guarantee is enforced by the compiler.

3 Provenance: who built it, and how

3.1 Authorship and ownership

Jounit was conceived, specified, implemented and documented by a single author, Swamy Gadila, and is published under a permissive dual licence (`MIT OR Apache-2.0`). The author has disclaimed individual ownership of the project: the code and the name are community property in the Bitcoin sense [13, 15]. The trademark treatment is deliberately separate from the licence — the name and ticker are community marks that anyone may use to refer to the project factually, but a fork must take a different name and a different ticker, so that two chains cannot both display `JNT`. Section 11 discusses this in full.

3.2 How the protocol was developed

The development method is itself part of the result, because the protocol's reliability rests on it more than on any single cryptographic choice. Three phases ran in parallel from the beginning.

Specification first. A single normative document (`SPEC.md`, roughly 2400 lines across 21 top-level sections) defines every wire format, every consensus rule, every fee and every domain separator, together with hand-written test vectors. The project's operating rule is that where a disagreement exists between the specification and the code, the specification wins — which is only meaningful because the specification is written to be executable in prose.

A decisions log. A parallel document records numbered decisions (D1–D24, plus P-series and M-series entries) with the reasoning behind each one, including the decisions that were reversed and why. Several of the most important properties of the finished protocol are direct consequences of decisions recorded here: a rate-anchored issuance rule instead of a stock cap, a stake-derived validator committee instead of a declared one, a proportional settlement fee instead of a flat one, and a post-quantum succession rule that is age-gated so that history is never stranded [19].

Independent re-implementation. A second implementation of the consensus-critical primitives — the proof-of-work digests, the wire format, the supply schedule and the difficulty rule — was written in Python directly *from the specification*, not from the Rust source, and is used to cross-check the Rust implementation. Agreement between two implementations written from two different descriptions of the same rules is evidence that the rules are unambiguous rather than merely self-consistent.

3.3 The remediation culture

The repository's verification record [16] documents the occasions on which a claim did not survive measurement. An earlier benchmark document was deleted because it attributed numbers to a machine that no probe had produced; an earlier fuzzing table was deleted as fabricated and replaced with a measured campaign; eight mismatches between the specification and the code were found and corrected. A protocol that is pre-launch can afford this kind of honesty, and a protocol that is post-launch cannot. The paper therefore treats the audit history (Section 9.8) as a first-class part of the result.

4 Architecture and implementation

4.1 Crate layout

The implementation is a single Rust workspace with seven crates [13].

Crate	Role	Notable contents
`jnt-core`	Consensus	chain, blocks, ledger, PoW puzzle, difficulty, supply, fees, bonds, finality, BLS, signatures, escrow, credit, L1/L2 bridge
`jnt-l2`	Layer 2	task and service contracts, disputes, deposits, exits, verified work certificates, replay
`jnt-net`	Networking	framing, handshake, messages, relay, sync, address book, bans
`jnt-store`	Persistence	append-only checksummed block log, metadata file, node-local state container
`jnt-node`	Node	CLI, daemon, REST API, wallet file, mediator
`jnt-worker`	Worker SDK	discovery, claim, execute, submit, dispute, exit; client only
`jnt-mcp`	Agent surface	keyless dispatcher over identity, payments, receipts

Table 1: Workspace layout. All consensus logic lives in `jnt-core`.

The architectural invariant that shapes everything else is that `jnt-core` contains *all* consensus logic and has no I/O, no clock and no network access. It is a pure state machine: given a block and the current state, it returns a new state or an error. This is what allows the Python cross-checks, the fuzz harness and the property-based test suites to exercise the real consensus path rather than a mock of it. The same discipline governs the policy layer, which is documented as pure and deterministic so that a wallet and a service that reach the same inputs reach the same answer.

4.2 Concurrency model

The node deliberately avoids an asynchronous runtime. A single thread owns the chain; peer threads exchange messages with it through channels. This is not an oversight: mining is an unbounded CPU loop, and sharing the chain under a mutex across a thread pool invites lock contention exactly when the node is busiest. The release profile additionally sets `lto = "thin"`, `codegen-units = 1`, `panic = "abort"` and, importantly, `overflow-checks = true`, so an arithmetic overflow aborts the process rather than wrapping silently. The protocol crates contain no `unsafe` blocks at all.

4.3 Storage and crash semantics

Two storage artefacts exist and are deliberately kept apart.

The chain store is an append-only log of checksummed blocks plus an atomically replaced metadata file. The crash semantics are explicit: a torn tail is dropped *and reported*, whereas damage to committed data causes the store to refuse to open rather than to silently truncate history.

The node-local books are written to a separate file, `state.bin`, using a simple container format: a magic value, a version, a length, a SHA-256 digest of the payload, and the payload itself. The file is written atomically and loaded fail-closed. The rationale is stated in the source: a node that cannot read its own books must refuse to start, because "I forgot every credit line" and "I have no credit lines" are the same bytes and opposite facts. The envelope version has advanced four times as features were added — settlement links, escrow-to-exit bindings, and finally the credit-attestation replay guard — and each bump is visible in the revision history.

4.4 Continuous integration

The continuous-integration pipeline enforces, on every change: formatting, `clippy` with warnings escalated to errors, the full workspace test suite, a release build, cross-checks of the consensus vectors against the Python reference, the attack simulations, and a short fuzzing pass over every registered target. A separate weekly workflow runs dependency auditing and licence-policy checks. All third-party actions are pinned to commit hashes, and automatic dependency updates are configured to ignore moving references and semver-major bumps of the cryptography crates specifically, so that a routine dependency bump cannot silently change a hash function or a signature implementation.

5 The blockchain layer

5.1 Encoding and identity

The block header is exactly 177 bytes and contains a version, a height, a 64-byte previous-hash pointer, a Merkle root, a millisecond timestamp, a base fee, a 128-bit difficulty, a nonce, a puzzle identifier, an epoch identifier, a 16-byte extra nonce and a one-byte signature-scheme tag. Transaction outputs are 28 bytes: an 8-byte little-endian value and a 20-byte address.

Hashing is split by purpose. Internal commitments, Merkle nodes and proof-of-work evidence use SHA-256 [20]. The two values an attacker most wants to forge — a transaction identifier and a block identifier — use SHA-512, on the explicit grounds that a 256-bit digest retains only about 128 bits of preimage resistance against a Grover-equipped adversary and about 85 bits of collision resistance against a quantum collision search, while doubling the output width restores a full 256-bit classical and quantum margin [21, 22]. Every hash input is domain-separated: distinct tags separate block hashing, transaction hashing, transaction-body signing, address derivation and each stage of the proof of work, so that a digest computed in one context can never be replayed into another.

The Merkle tree separates its leaves by type as well. Layer-1 leaves, Layer-2 leaves, the Layer-2 state root and exit claims each carry distinct domain tags, so an attacker cannot present a Layer-2 payload as a Layer-1 transaction or lift a state root into a transaction position.

5.2 The hybrid proof of work

The proof of work is a three-stage puzzle, specified in Section 5 of the protocol specification and implemented in the `puzzle` module [14].

Stage 1 — the SHA-256 chain. The header is serialised and hashed with a domain tag and a 64-bit nonce; the result is then iterated for a fixed number of rounds, each round hashing the previous value together with a round index. This stage carries the security of the system. It is the part a quantum adversary must break.

Stage 2 — the integer-only neural network. A small feed-forward network with three layers of sixteen hidden units is evaluated on the Stage-1 output. The weights are not stored anywhere: they are derived from the epoch identifier by hashing, which means no miner can precompute them before an epoch begins, and every miner in the same epoch computes the same function. The accumulator is a 64-bit signed integer that wraps to 32 bits at each layer, and the activation is a shift-and-clamp. The bound on the accumulator is derived in the specification and holds by construction, so no floating-point rounding difference can arise between implementations.

The neural stage is a *cost model*, not the load-bearing part of the proof, and two mechanisms enforce that. The first is a work-share band: the registry entry must declare that the neural stage accounts for between 30% and 50% of modelled work, and an entry outside that band is rejected outright, so the neural stage can never become the weak link it exists to make useful. The second is the ordering of the stages: the security-critical hash is Stage 1, and Stage 3 binds the Stage-1 and Stage-2 outputs together with the nonce and epoch identifier into the single value that must fall below the target.

Stage 3 — binding. The final digest hashes the domain tag, the epoch seed, a digest of the activations, the nonce and the epoch identifier. Validity is the usual comparison against a target derived from the block's declared difficulty:

$$\mathrm{U256BE}\big(\mathrm{SHA256}(D_{S3} \parallel \mathrm{seed} \parallel d_{\mathrm{act}} \parallel \mathrm{nonce} \parallel \mathrm{epoch})\big) \le \left\lfloor \frac{2^{256}-1}{d} \right\rfloor, \tag{1}$$

where d is the block difficulty.

A second proof-of-work family, a memory-hard scratchpad puzzle, has been specified and implemented and has passed a scoped internal security review [23], but it is deliberately *not registered* as an active puzzle identifier: an unregistered puzzle validates nothing, so the alternative cannot silently displace the deployed one.

5.3 Difficulty adjustment

Difficulty follows a windowed, linear-weighted moving average in the style established by the CryptoNote work [7], with a window of sixty blocks and a maximum per-adjustment change of ten per cent. The retarget is multiplicative rather than additive:

$$d_{\mathrm{new}} = d_{\mathrm{prev}} \cdot \frac{T}{\bar{t}}, \tag{2}$$

with T the target block time and $\bar{t}$ the observed average solve time. Additive adjustment would bias the schedule in one direction under sustained pressure, and multiplicative adjustment has the additional property that the relative change is bounded by construction.

Timestamps are checked against a lower bound only: the median of the previous eleven blocks' timestamps. There is no consensus upper bound, because a clock rule in the consensus path is a lever that a miner can pull to influence difficulty; instead the node applies a local sanity limit one block time beyond the target.

5.4 Fork choice

The chain is linear and hash-linked; there is no DAG and no uncle structure. Fork choice is heaviest cumulative work: the node keeps the branch with the greatest accumulated difficulty, with ties broken deterministically by lower height and then by the lowest block hash, so that two nodes holding the same set of valid blocks always agree on the tip. Cumulative work is computed incrementally at insertion time by saturating addition, and side branches are pruned once they exceed a threshold.

Because the fork-choice rule is a total order over the set of valid blocks, the system inherits the security analysis of cumulative-work chains [24, 25]: an attacker must spend real work to displace the honest branch, and the cost of a reorganisation grows with the work it must discard.

5.5 Fees, burn and the protocol reserve

The fee market is a base-fee mechanism in the style of `EIP-1559` [26]: a required fee proportional to gas, a base fee that rises when blocks are full and decays otherwise, clamped to a configured floor and ceiling. Gas is charged per transaction, per input and per output.

The base fee is split in half: one half is burned, the other half is credited to an on-chain reserve used for protocol security and compute budgeting. The split is executed with integer arithmetic that assigns the odd satoshi to the burned side, on the reasoning that a burn is irrecoverable and conservative rounding in that direction is the safe choice. Only the tip above the base fee is paid to the miner.

5.6 Issuance

There is no stock cap. Issuance is rate-anchored and time-based [27], and three properties of the schedule are worth stating because they are unusual and because they are enforced in code rather than in prose.

Zero premine. The genesis block mints nothing. This is enforced by the code path, not merely by the absence of a special case.

A halving ladder in whole units. The block reward halves every 262 800 blocks — approximately five years at a ten-minute block time — in whole units of the token rather than in shifted subunits, because halving 9.5 to 4.75 breaks the arithmetic. The ladder is

$$76,\ 38,\ 19,\ 9,\ 4,\ 2,\ 1\ \ \text{JNT}, \tag{3}$$

and the total issuance over the ladder era is exactly 39 157 124 JNT. That figure is not the geometric shortcut: the genesis block occupies a rung-zero height without minting, so the first rung is paid by one fewer block than a naive computation suggests, and the difference is precisely the shortfall between the approximation and the chain's real reach.

A tail that never stops. After the ladder is exhausted, the reward decays to a floor rather than to zero: 0.50, 0.25, 0.125 JNT, and then a permanent floor of 0.1 JNT per block, forever. Issuance therefore never terminates, and net supply can actually fall, because the burned half of the base fee removes supply from the spendable set. The protocol's own monetary analysis reports that net supply crosses below break-even at roughly five per cent activity and sits at about a quarter of break-even at the fee floor. This is a measured consequence of a deliberate design, not an artefact.

Two bounds enforce the schedule independently of the miner, the validator and the node, all checked when a block is applied. The first is an issuance integral: a block may not mint past the cumulative issuance at its height, and overflow is refused rather than wrapped. The second is a rate anchor: after ladder exhaustion, gross issuance inside the current halving window may not exceed one per cent of the supply issued at that window's start, which bounds dilution at roughly 0.2 per cent per year in the limit. The approved schedule sits about three times under the anchor, so it never binds today; it exists to refuse a *future* schedule that would dilute far faster.

5.7 Finality

Jounit uses two layers of finality, and the distinction is important because the two protect against different events.

Depth-based finality is the familiar rule: a transaction buried under a configured number of blocks is considered settled. On the mainnet profile this is one hundred blocks. It is cheap and probabilistic.

Binding finality is a quorum certificate over a block height, signed by a stake-weighted committee using BLS signatures [28, 29]. A checkpoint is accepted only if it satisfies a seven-clause checklist: epoch alignment, a self-derived signer set, an exactly-sized signer bitmap with no bits outside the set, a popcount equal to the declared signer count, satisfaction of a quorum threshold, verification of the BLS aggregate signature, and the presence of a post-quantum sidecar where one is required. A certificate at height H binds the chain at H only if the canonical block at H exists, the certificate validates, the quorum is met, the height is strictly greater than the previous binding point, and no conflicting certificate exists for the same height.

The quorum threshold is defined as a ceiling of two thirds:

$$q(n) = \left\lceil \frac{n \cdot 6667}{10000} \right\rceil , \tag{4}$$

giving three of four and fifteen of twenty-one for the relevant committee sizes. The committee is derived locally from the bond ledger by stake-weighted selection — never from a list a peer supplies — so a message is always recorded against the receiving node's own view of the committee and height, rather than against the sender's say-so.

Equivocation is detected structurally: a conflicting pair of certificates from the same signer is admissible evidence, is bounded in size, and can be submitted permissionlessly. Reorganisations that would cross the binding finalised point are refused outright, so a finalised history cannot be undone.

5.8 Peer-to-peer layer

The transport is TCP, with one thread per peer and no asynchronous runtime. Framing is `magic` ‖ `type` ‖ `length` ‖ `payload`; the magic value is re-checked on every frame rather than only at connect, the payload length is bounded before any allocation is made, and an oversize frame closes the connection rather than being skipped, because a stream protocol has no resynchronisation point. Payloads are read incrementally so that a lying length field pins only the bytes actually delivered.

The handshake exchanges a network magic, a network identifier, a protocol version, a service bitmask, the peer's height and tip hash, a user agent and a nonce, and the responder must echo the initiator's nonce. This is *transport-level* peer identity only: there is no node key and no authentication in the current version, and the protocol specification says so rather than implying more.

Relay is bounded on every axis that a flood could exploit: the number of inventory items per message, the number of transactions requested per batch, the number of in-flight and pending relay items, the number of known transactions remembered, and separate per-peer per-minute budgets for transaction relay and for control messages. A node never relays a transaction it did not itself admit, and never relays a block it has not validated.

Misbehaviour is scored rather than instantly banned: a ban score accumulates, decays at one point per hour, and a threshold bans the peer. A block that fails validation costs a small score increment rather than an outright ban, because a peer relaying one bad block is not the same as a peer attacking the node. Connection counts are capped (125 peers, of which at most 32 inbound), and in-flight handshakes are counted separately, since a handshake that has begun but not completed is invisible to the peer count and would otherwise let a single host pin one thread and one socket per connection.

Finally, there is no peer discovery protocol at all: no address gossip, no DNS seeds, no super-nodes. Bootnodes and a local peer file are the only peer sources. This is a scoping decision, recorded as such, and it means that the network's resistance to partitioning rests on the operator rather than on the protocol — a limitation we prefer to state rather than hide.

Code	Type	Purpose
`0x10`	Post	Submitter opens a task, locks the escrow, publishes the compute specification
`0x11`	Claim	Worker takes the task and posts a deposit
`0x12`	Submit	Worker submits the result hash with a signed attestation
`0x13`	Dispute	Anyone challenges the result with a counter-result and a deposit
`0x14`	Settle	Committee applies the decision: pay, refund or slash
`0x15`	ExitQueue	Owner starts withdrawing value back to Layer 1

Table 2: The six Layer-2 transaction types. A task moves through Open $\rightarrow$ InProgress $\rightarrow$ ResultSubmitted $\rightarrow$ Disputed? $\rightarrow$ Settled, with Expired as the timeout branch.

6 The task economy

6.1 Design

Layer 2 is not a second consensus protocol. Layer-2 transactions are carried inside Layer-1 blocks and committed by the block's Merkle root, and Layer-2 state is rebuilt from that chain by deterministic replay. What does not exist is any rule that makes a Layer-1 block invalid because of its Layer-2 content: a Layer-2 payload that fails to apply is recorded by the replayer and skipped deterministically, never converted into a consensus rule. This is a deliberate separation — Layer 2 inherits Layer 1's ordering and immutability without inheriting Layer 1's latency, and the failure mode of a malformed Layer-2 payload is a skipped payload rather than a halted chain.

The vocabulary of the task economy is six transaction types.

6.2 Value conservation

Every Layer-2 settlement enumerates the movement of every unit of value — worker paid, worker deposit returned, worker deposit slashed, submitter refunded, challenger refunded, challenger deposit burned, worker deposit burned, fee burned, fee to the reserve — so that a settlement cannot be under-specified in a way that creates or destroys value. The Layer-2 state machine asserts a conservation invariant after every step:

$$\sum \text{accounts} + \sum \text{locked} + \text{burned} + \text{reserve} + \text{vault remaining} = \sum \text{credited}, \tag{5}$$

and a fuzz target asserts exactly this property over adversarially generated transaction sequences. The joint Layer-1/Layer-2 check needs no exit term at all: the Layer-1 payment and the Layer-2 debit cancel while the sink term falls by the same amount.

6.3 Deposits and exits: the bridge between layers

Deposits are not one-way, and the mechanism by which value returns to Layer 1 is the most security-sensitive part of the design.

Deposit. A deposit is an ordinary Layer-1 payment to the profile's derived sink address — no new transaction type, no special opcode. The amount is read from the committed output, never from the claim, and the deposit identity is the pair (transaction identifier, output index). Credit requires both depth-based finality and binding finality, so the rule for maturity is never weaker than the rule used elsewhere in the protocol.

Queue. The owner signs an exit-queue transaction. It debits the owner by amount plus exit fee plus queue fee, and records $(\text{owner}, \text{nonce}) \rightarrow (\text{amount}, \text{destination}, \text{exit fee}, \text{queued height})$. Nonces are per owner and strictly increasing; zero amounts, under-floor fees and repeated or rewound nonces are refused.

Maturity. An exit becomes settleable after the same finality requirement as a deposit — depth and binding — never less.

Release. The claim is authorised by a Merkle inclusion proof against the Layer-2 state root committed at the queue block, not by a signature, because the sink holds no key. The verifier recomputes the exit record leaf from the recorded fields and checks it against the committed root; no caller-supplied amount, destination or fee is ever read from the claim. The inputs consumed must sum to exactly the amount plus fees, with no change and no tip, so an exit whose funding is

Kind	What backs it	Offline-verifiable?
`Quorum`	The settle payload's own BLS aggregate over the decision	Yes, against the validator set
`WindowElapsed`	Nothing beyond the on-chain record that nobody objected in time	No

Table 3: Two kinds of verified work certificate. The weak basis is unmissable rather than a convention a relying party could forget: the API refuses the window-elapsed kind by name when a correctness claim is required.

not present stays queued rather than draining a stranger's deposit. Consensus pays only the destination that the proven record names, so a caller cannot redirect it, and the crank collects a fee the owner prepaid at queue time — which is what makes a crank nobody wants to run a non-issue rather than a griefing vector. The crank is permissionless: a read-only endpoint hands settleable claims to any third party, so a non-miner can construct the block that includes them.

Reorganisation safety. A release is an ordinary undoable Layer-1 operation. Rolling back the settling block restores the sink outputs, clears the exit identity, and Layer-2 replay regenerates the watch. The exit identity is consumed in consensus state, so a given $(\text{owner}, \text{nonce})$ pair is released exactly once, ever.

6.4 Disputes and adjudication

The interesting security question in a task market is: who decides, and can the decider be influenced? Jounit's answer is structural.

The reference cannot be supplied. The canonical reference result that decides a fraud dispute is computed by the arbitration engine from the task's own committed fields. It is never accepted as an argument. No caller can dictate a verdict, because there is no parameter through which a verdict could be dictated.

Evidence and verdicts. A dispute carries evidence — a counter-result and the associated attestations — and the engine produces exactly one of three verdicts: the worker was correct, the challenger was correct, or the dispute is inconclusive. Inconclusive is deliberately not a verdict: it settles nothing, slashes nothing and burns nothing, because a system that must burn something when it cannot tell would punish honest disagreement.

Consequences of a fraud verdict. The worker's deposit is moved to the vault, where half is burned immediately and half vests linearly over a year, funding a whistleblower bounty of twenty-five per cent of the slashed amount paid out of the reserve. The submitter is refunded in full; the challenger is refunded and receives the bounty. No certificate is issued for a fraudulent or expired task.

Committee voting. Votes are transported over the peer-to-peer network as arbitration votes and tallied against a bounded book. Every vote is checked against the receiving node's own derived committee, and votes are recorded for the height they concern rather than for the height of the sender's claim. The committee meets a quorum of two thirds for a decision to bind.

6.5 Verified Work Certificates

A verified work certificate is the protocol's receipt for work that settled as correct. It records that task T settled correct with result R, worker W was paid P at height H.

It is a *derived consequence* of a settlement, not a claim an actor makes. There is no issuance transaction and no issuer key, because the settlement path already performed all the authorising. Its identifier is derived from every other field, so a tampered certificate cannot carry a matching identifier, and its encoding is byte-unique.

Two kinds exist, and the difference between them is load-bearing:

The unchallenged-result rule deserves emphasis because it is the honest weak point of any optimistic settlement system. If a worker submits a result and nobody challenges it within the dispute window, the task auto-settles correct and the worker is paid. That is correct protocol behaviour: a challenge must be able to overturn a settlement, because otherwise anyone could file a dispute and never resolve it, stalling every task forever. The consequence is that an incorrect result that nobody challenges in time *is* paid. What must never happen is that such a payment being mistaken for verification. Three mechanisms prevent it: the certificate kind is explicit in the data structure, the validation function refuses the

weak kind by name, and the certificate assertion returns false for it — so a relying party that needs independent proof must demand the quorum basis, and there is no argument by which the weak one satisfies it.

6.6 Reputation

Reputation is a bounded accumulator per identity, driven by a fixed weight table held in code rather than in data: service completed $+1$, payment completed $+1$, verified work $+2$, dispute won $+1$, dispute lost -3. An event carries no amount, no weight and no signature of its own, so a differently-shaped event cannot forge a better score; and losing a dispute costs three times what winning one earns, so baseless challenges are expensive.

Critically, reputation is *not* a consensus input. A credit limit computed on-chain from a mutable score would make every reputation update a consensus event and make fresh identities farmable, which is the cheapest available attack against the gate. The credit module therefore exposes no function that accepts a score at all: the guarantee is the type system refusing the call, not the code declining it.

7 Agent authority and credit on chain

7.1 Why it had to move

The spending-policy layer of an earlier revision of this protocol was *node-local*. A policy constrained a payment on the node that issued it, and the chain never saw it: the consensus core contained no reference to a policy, an agent, a delegation or a credit at all. A machine fleet bound by policies was therefore bound by the goodwill of whichever node had issued them, and two nodes holding the same chain could disagree about whether a spend was permitted.

The credit facility had the same shape and two further problems. Depositing credited a local balance without moving a coin, and opening a position accepted a *lender* field that anybody could write: a loan the lender had never agreed to, on money that did not exist, visible only to the node that recorded it.

Both are now Layer-2 consensus state. The distinction that matters is not where the code lives but what a node is *required* to do with it: a rule that every node replays is a rule, and a rule one node checks is advice.

7.2 The grant

A grant is the owner's signed permission. It is published with a Layer-2 transaction, and the owner's signature covers three things at once — the grant bytes, the *spender* (the key that may exercise the grant) and the *agent identity* the grant names. That binding is the whole point: a grant hash on its own is a capability, and a capability anybody can exercise is worthless. Here the owner says *this key may spend under these limits*, and the chain remembers it.

The grant carries the limits the owner stated: a per-transaction cap, a tumbling per-window cap, an exhaustive recipient allow-list, an exhaustive service allow-list, a validity window, and a parent link for a delegated grant. The chain enforces all of them against state it keeps:

1. the per-transaction cap: $a \le C_{\mathrm{tx}}$;
2. the per-window cap, over a tumbling window that starts at the first spend and rolls over after W blocks:

$$\sum_{\text{spends in the window}} a \;\le\; C_{\mathrm{window}}; \tag{6}$$

3. the recipient allow-list, where an *empty* list allows nothing;
4. the service allow-list, where a grant that names services authorises no untagged spending;
5. the validity window, which is inclusive at its start and exclusive at its end;
6. a strictly increasing nonce, which is what makes an authorisation unreplayable;
7. and the rule that a height inside a grant never runs backwards.

One grant is live per agent. A second publication for the same agent is refused, so a second grant cannot be used to double the caps the first one grants. A delegated grant must *narrow* its parent — the child's caps are not wider, its allow-lists are not wider, and it does not outlive it — and the narrowing is checked against the parent's bytes as they are held on chain, not against a claim about them.

7.3 Revocation, and what it reaches

The owner can kill a grant, and the kill switch reaches every descendant: a spend checks the whole ancestry, so a delegated fleet dies with the grant it descends from, without a revocation transaction per link. The walk is bounded by the delegation depth cap and fails closed — a missing ancestor, a revoked one, or a chain deeper than the cap all mean *not live*.

7.4 Spending

A spend is signed by the spender, not by the owner. The owner signed once, at publication; the grant itself authorises every later spend. The money comes out of the grant owner's balance — a grant is an allowance over the owner's own funds — and the fee comes out of the same balance on top.

7.5 Credit

A loan is four transaction types. The *lender* signs the terms; the principal moves to the borrower's balance when the position opens; the borrower's own funds are locked as collateral, and they are checked *before* the principal arrives, so the borrowed money cannot secure itself. Interest is simple interest per whole window on the outstanding principal,

$$\text{interest} \;=\; \frac{\text{principal} \times r_{\text{bps}} \times \text{windows}}{10\,000}, \tag{7}$$

charged once per period rather than once per repayment, and a repayment pays the interest before the principal. A repayment larger than the debt is refused: the lender is paid the debt and not a satoshi more.

A default is permissionless — anyone may trigger the settlement of an overdue position, because a default only the lender could trigger is a default that could be delayed. The collateral answers for the debt, the surplus returns to the borrower, and what the collateral does not cover is a shortfall the lender absorbs rather than money the protocol invents.

7.6 The state commitment, and the invariant

Both families live in the Layer-2 state, under their own leaf tags, and are committed by the block's Layer-2 state root. The conservation invariant that already covered the task economy now covers them too:

$$\sum \text{accounts} + \sum \text{locked} + \sum \text{exits locked} + \sum \text{collateral locked} + \text{burned} + \text{reserve} + \text{vault remaining} \;=\; \sum \text{credited}, \tag{8}$$

asserted after every single transition, including every refusal.

7.7 How this is checked

The rules of this section are implemented twice: once in the protocol, and once in an independent reference written from the specification in a different language. The reference derives the addresses, the agent identities, the grant hashes and the position identifiers itself, and emits a vector file of scenarios; the protocol's test suite replays those scenarios and asserts the same verdict, the same reason and the same final state. A rule that refuses for a different reason is a different rule, and the two implementations have to agree on which reason it was.

8 Value, wallets and market operations

8.1 Keys and addresses

Jounit supports three wallet modes — classical, hybrid and post-quantum — and the address is derived from the *whole key set*, not from a single key. A hybrid bundle contains an ECDSA component and a Schnorr component; the post-quantum bundle adds an ML-DSA-65 component [30]. Because the address commits to the serialised bundle, a hybrid key set cannot quietly be replaced with a smaller one, and the three modes produce three distinct addresses by construction.

Two rules govern signature bundles and both are enforced in code: every component must verify, and the address commits to the whole key set. The second is what makes downgrade attacks against a hybrid signature impossible.

The post-quantum requirement is age-gated. It is off on every current profile, and when it is switched on it applies only to outputs created at or after the activation height, so that historical outputs are never stranded. This is a deliberate

reading of the general transition problem: the cost of migrating history is paid by nobody in particular, and the benefit of an early switch does not justify stranding coins [31].

8.2 Key management

Wallet keys are held in a file whose format is a magic value, a random salt, a random nonce and a ChaCha20-Poly1305 ciphertext [32], with the key-encryption key derived from an operator passphrase by Argon2id [33]. The file is written atomically: a temporary file is created with owner-only permissions *before* any key material is written to it, flushed, and renamed over the destination. A fresh salt and nonce are generated on every write. The passphrase is read from an environment variable rather than a command-line flag, because flags are visible in the process table. Storing keys in plaintext requires an explicit opt-in flag; the default is refusal.

Secret hygiene is enforced at the type level as well: the wallet's private key is wrapped in a zeroising type, the debug representation of a wallet redacts the scalar, and the value is zeroised on drop.

8.3 Storage and transfer of value

Value is stored in an unspent transaction output model. Each output is a 28-byte structure; a transaction consumes previous outputs and creates new ones; coinbase outputs mature after a configured number of blocks, and a miner cannot claim two matured rewards in one block because outputs to the same address accumulate rather than replace.

Transfers are ordinary: the node can construct and broadcast a transaction locally or over the peer-to-peer network, and the CLI reports balances in whole tokens while accepting amounts in subunits. A dedicated verification command replays the entire chain from genesis and diffs the resulting output set, supply, bond ledger and binding-finalised point against stored state — the check worth running after any change to the protocol.

8.4 Buying and selling: the payment path

There are two payment paths, and it is worth being precise about which is which. The *authority* path — the one this paper's Section 7 is about — moves real value under a limit the chain enforces. The *service* path below is the operational bookkeeping around it: a request, an authorisation, a receipt and a settlement link. The service path never invents balance: a settlement still needs a transaction that actually pays.

For a machine-to-machine transaction, Jounit separates *authorisation* from *payment*. The separation is what allows an agent to act without holding a spending key in the general case.

Authorisation. A principal signs a spending policy. The policy answers exactly one question: may this agent, at this height, spend this much, to this recipient? The implementation is pure and deterministic, and every refusal is a named error rather than a boolean. The ambiguous cases fail closed: an empty recipient list authorises nothing rather than anything; an empty service list authorises no service-tagged payment; a nonce must strictly increase, so an authorisation cannot be replayed; a height below the policy's own window start refuses, so time does not run backwards inside a policy; and cumulative spend is bounded per window, with the window rolling over rather than accumulating forever. The same rules are the ones the chain enforces for an on-chain grant, so an operator can reason about one set of limits rather than two.

Attenuated delegation. Delegation chains let an agent grant a subset of its own authority downward. Each level inherits a strict subset of its parent's policy — never a wider one — so authority narrows as it travels. A payment under a chain names its own agent identifier, which is derived from the ancestry it presents rather than from the payer's address; the node records the links so that the lineage can be proved later, verifies the owner's signature on every policy it grants under, and propagates a revocation down to every recorded descendant. On chain, the same rule is checked against the parent's bytes as the chain holds them (Section 7), so a delegated fleet's authority narrows for every node, not only for the one that issued the grants. The practical result is a fleet of agents whose operator can bound what any single agent can spend without re-authorising each transaction by hand.

Payment. A machine payment carries the authorisation, the recipient, the amount and the reference. A receipt book records deterministic service receipts and refuses a receipt that conflicts with an existing one for the same payment. A payment tracker provides exact-payment replay protection and refuses rather than evicts when its cap is reached.

Settlement. A payment is not settled merely because it was recorded. A settlement endpoint binds the recorded payment to a Layer-1 transaction that pays its recipient; a transaction that does not pay the recipient is refused, a second,

different transaction is refused as a conflict, and the link persists across restarts. This is the mechanism that prevents a node from claiming that a payment happened without the coins having moved — the same concern that motivates transaction inclusion proofs elsewhere in cryptocurrency design.

Streaming micro-payments. A payment stream is pay-as-you-use: a payer-signed rate per block, a window, a hard cap, a recipient and the governing policy for non-human payers. Settlement receipts are payer-signed, the stream book is fail-closed (exact amounts, strictly ordered settlements, no future-dated settlements, accrual checked for wraparound), and settlement rides the ordinary payment path with the stream identifier as the reference, so no new transaction type is introduced.

8.5 Credit: borrowing against future work

An agent with a demonstrated track record should not have to pre-fund every transaction. The credit facility allows an agent to borrow against future work. The loan itself is the on-chain facility of Section 7: the lender signs, the principal moves, the collateral is the borrower's own money, and the interest and the default rules are the chain's. What remains here is the service layer around it, and it is worth being explicit that it is a *gate*, not a source of spendable balance.

Issuance is attested, not open. New credit enters only through an attestation signed by a key the operator listed as trusted. With no key listed, every mint is refused — the default is refusal, not implicit trust. An attestation binds the network, the subject, the amount, an issue height, an expiry height, the issuer key and the signature; it is checked against the network, the height window, the trusted issuer list, the signature itself, and a replay guard that records consumed attestations and persists across restarts, so one attestation mints exactly once, ever.

Credit is always collateralised. The required collateral is a function of the amount, discounted by a posted bond but never replaced by it, with a minimum collateral ratio enforced. On chain, where no bond is posted, the requirement is full collateralisation. Settlement takes the position's own collateral first, then the bond, and *never* reputation. Collateral in excess of the debt returns to the borrower: a default takes what it is owed and no more. Any residual debt is a reported shortfall that the lender absorbs, rather than the protocol inventing funds. A position runs to a deadline, and before that deadline a default is refused — a lender cannot settle a merely late position on a whim — and the bond must be posted before the position opens, so no discount is ever unbacked.

One implementation of each rule. The collateral arithmetic and the interest arithmetic live in one place, and the on-chain facility and the service-layer book both use them. Two implementations of the same rule would be two rules.

8.6 Conditional escrow

Conditional escrow is the general primitive behind the task economy, and it also exists standalone for non-compute deliverables.

A contract-selected arbitrator. Each escrow names its own arbitrator as a BLS public key, bound when the escrow opens. There is no committee, no central arbitrator and no on-chain registry: the contract itself is the registry. The escrow identity is a hash over the network, the parties and their keys, the arbitrator, the amount, a nonce and the condition, so two escrows between the same parties with different terms are distinct objects.

Release modes. An arbiter release is verified as exactly one BLS signature under the escrow's own arbitrator key; a mutual release requires exactly two, and the claimed keys must equal the escrow's bound payer and payee keys, so a release cannot be forged by a party that is not the party.

Expiry and refund. Every escrow expires after a configured number of blocks — seven days on the mainnet profile — and refunds its payer. A refund requires no signature at all, deliberately, because it is the path for "nobody will ever sign"; releases are refused after expiry, so a late signature cannot race a refund.

Oracle conditions are refused. Conditions that genuinely require an oracle are refused when the escrow opens, because this codebase implements no oracle verifier: an oracle-gated escrow would be a contract that nobody could ever satisfy. Failing closed here is the difference between a mechanism that is unavailable and a mechanism that is unsound.

Settlement to the chain. A released escrow does not merely change a book entry. The settlement endpoint binds the release to the Layer-1 exit that pays the payee — owner is the payer, destination is the payee, amount is the escrow's

Operation	Fee
Post task	5 000 subunits
Claim task	2 000 subunits
Submit result	2 000 subunits
Raise dispute	10 000 subunits
Settle	$\max(0.1\% \times \text{value},\ 1\,000)$ subunits
Queue exit	2 000 subunits
Exit fee floor	1 000 subunits
Layer-2 deposit floor	1 000 000 subunits per sink output

Table 4: Layer-2 fee schedule. A proportional settlement fee means that a proportional cost follows the value released, while a floor keeps small settlements from being free to spam.

Component	Trusted?	Reason
Consensus core	Yes	The only trusted code; it has no I/O, clock or network
Peers	No	All rules are checked locally
Block producer	No	A lying Layer-2 root is accepted but only stalls exits
Exit claimant	No	The crank is permissionless and pays only the recorded destination
API caller	No	The API never signs and never submits to consensus
Model Context Protocol surface	No	It verifies with the same public functions; it cannot mint authority
Worker SDK	No	Every privileged transition is a signed Layer-2 transaction the protocol verifies
Reputation	No	Not a consensus input, by construction
Delegation service layer	No	The lineage book is the node's own record; the narrowing rule itself is a consensus rule (Section 7)
Credit issuer key	Yes, locally	An explicitly configured operator-level trust anchor, and only for the service-layer book
Grant owner	Yes, by construction	A grant is the owner's own signature, and the chain enforces only what that signature says

Table 5: Trust boundaries. Note that the two "yes" entries are narrow: one is the protocol's own code, the other is a key the operator chooses to trust and can therefore choose not to trust at all.

amount — and queues that exit through the ordinary withdrawal path. The escrow reads finalised only once consensus actually released the bound exit; a height stamp alone no longer finalises a bound escrow. This means the escrow and the exit cannot disagree, because the escrow's final state is defined by a Layer-1 fact rather than by a local one.

8.7 Fee schedule

Layer-2 fees use their own path and never consult the Layer-1 base fee, with the flat fees shown below and a settlement fee proportional to the value released (one tenth of one per cent, with a floor).

9 Security model

9.1 Threat model

The adversaries considered are the ones a public, permissionless, TCP-hosted network actually faces: a remote attacker who wants to steal funds; a peer who wants to exhaust a node's resources; a miner who wants to commit a block that should be rejected; a block producer who wants to forge an exit claim; a worker who wants to be paid for work it did not do; a submitter who wants a refund without paying; a challenger who wants to grief a competitor; an API client who wants to make the node misbehave or leak secrets; and a local attacker who can read the node's data directory.

9.2 What is trusted

The trust model is best stated as a table, because the interesting entries are the negative ones.

Primitive	Where	Purpose
SHA-256	everywhere	commitments, Merkle nodes, proof-of-work stage 1, stage domain separation
SHA-512	block and transaction identity	256-bit classical and quantum margin on the two forgeable values
secp256k1 (ECDSA, Schnorr)	wallet and spending keys	transaction signing, hybrid bundles
BLS12-381	checkpoints, escrow, arbitration, credit attestations	quorum certificates and threshold aggregation
ML-DSA-65	checkpoint sidecar, post-quantum mode	post-quantum signatures without pairing aggregation
Argon2id	wallet file	passphrase-based key derivation
ChaCha20-Poly1305	wallet file, node state container	authenticated encryption

Table 6: Primitive inventory. The BLS implementation is a reference pairing library with published audit history [34]; the aggregate signature scheme follows the standard construction and its known rogue-key attacks [28, 35].

The absence of a trusted issuer by default is worth emphasising: credit minting is fail-closed, so an operator who does not configure an issuer has a node that refuses every credit request. The alternative — an implicit issuer — would mean the operator had trusted something they never chose.

9.3 Cryptographic mechanisms

Because BLS aggregation has no pairing-based analogue in the lattice setting, the post-quantum signature travels in a sidecar: a bounded list of per-signer lattice signatures accompanying the checkpoint. A signer bitmap that holds a registered lattice key but presents no sidecar entry is refused — a sidecar stripped in transit lands as "missing", never as "valid without".

9.4 Consensus-level security

Integer-only arithmetic. Every consensus computation is integral. Difficulty, supply, fees and the proof-of-work forward pass all refuse overflow rather than wrapping it, and the release profile enables overflow checks in the compiler.

Ordering of checks. A submitted block is checked for parent existence and for agreement with the difficulty its own branch demands *before* the proof of work is verified. This ordering is not cosmetic: a header that declares a trivial difficulty has a trivially satisfiable target but still costs the verifier a full proof check, so checking the difficulty after the proof would let anyone burn the node's CPU for free, repeatedly. The fix was applied across the consensus, peer-to-peer, API, command-line and Model Context Protocol surfaces in a single audit pass.

Reorganisation bounds. An attempted reorganisation that would cross the binding finalised point is refused, and the node state is restored.

Proof-of-work diversity. The alternative memory-hard puzzle is specified, implemented, separately reviewed and left unregistered, so it cannot affect consensus until it is deliberately enabled.

9.5 Network and API security

The peer-to-peer layer bounds every quantity an attacker can inflate: frame payload size, inventory items per message, transaction request batch size, in-flight and pending relay items, remembered transactions, per-peer per-minute transaction and control budgets, total and inbound peer counts, in-flight handshakes, and the number of arbitration tallies remembered.

The HTTP API enforces, on every bind: a request-body size limit; a cap on concurrent requests in flight; a per-request deadline; a separate deadline for the client to finish sending request headers, which is the slowloris bound; a connection cap enforced in a manual accept loop so that an over-cap socket is dropped before a single byte is read; a host-header check that refuses a missing, unknown or bracket-malformed host on every bind including loopback, which is a defence against DNS rebinding; and, on any non-loopback bind, a mandatory bearer token compared in constant time over hashes so that neither the value nor its length leaks through timing. Non-loopback binds are refused outright if no token

Surface	Guard	Behaviour on replay
Payment	exact-payment tracker, capped	refuse (never evict)
Receipt	receipt book, contradiction detection	refuse by name
Credit attestation	consumed-attestation set, persisted	refuse as reuse
Credit reservation	reservation log, idempotent	no second reservation
Policy authorisation	strictly increasing nonce	refuse
Settlement link	one payment, one transaction	refuse as already settled
Escrow-to-exit binding	one escrow, one exit	refuse as already settled
Exit identity	$(\text{owner}, \text{nonce})$ consumed set	release exactly once, ever
State file	SHA-256 digest, fail-closed load	refuse to start

Table 7: Replay guards. The general policy is refusal rather than eviction: dropping the oldest entry would silently re-open an old replay window.

is configured. Error responses are uniformly JSON with a stable machine-readable code and a human-readable detail string, and the detail never contains signing material. Rate limiting is a token bucket per remote address, with IPv6 addresses grouped by prefix.

9.6 Replay protection and fail-closed behaviour

9.7 The lying-root rule

One security property deserves a dedicated subsection because it is unusual and because it demonstrates the design's threat model.

The Layer-2 state root committed by a block is checked for *presence*, not for correctness. A block that commits a root inconsistent with the Layer-2 state the proposer actually computed is accepted at Layer 1. The consequence is bounded and one-way: exits whose inclusion proofs do not verify against the committed root simply never become settleable, so they stall. They do not pay the wrong party, they do not pay twice, and they do not corrupt the Layer-1 ledger. Splitting proof-checking from chain-checking is what makes a forged proof cryptographically impossible even when the block producer commits a lying root.

This is a deliberate trade. Requiring every node to re-execute the entire Layer-2 state machine to validate a block would make Layer 2 a consensus layer, with all the latency and reorganisation costs that implies. Instead, the cost of a dishonest producer is paid by the honest users of the exit queue whose exits stall — and by the producer, whose own exits stall too. A test suite mines blocks past the finality depth to demonstrate that the stall is permanent rather than merely immature.

9.8 Security engineering as a process

Security in this project is treated as a measurable process rather than a claim.

Four parallel audits. A dedicated audit pass covered the API, the wallet and command-line surfaces, the peer-to-peer layer, and the Layer-2 and agent surfaces in parallel, producing a set of fixes applied together. The findings were, in outline: the consensus check-ordering problem described above; an unbounded peer-event queue replaced with a bounded one that pushes backpressure onto the sender instead of buffering; the in-flight handshake cap; per-peer rate limits on control messages, which are pairing-checked on arrival and would otherwise burn CPU in a flood; a cap on remembered arbitration tallies; a capacity check moved ahead of deduplication scans in the inventory handlers; length-hiding token comparison; IPv6 rate-limit buckets keyed by prefix; refusal of empty-token public binds; saturating arithmetic in the transaction builder; an off-by-one in the spendable-balance calculation; path traversal through a validator key label; a queue drain that only happened while mining; refusal to overwrite an existing wallet file; and errors during shutdown flush that were previously discarded.

Red-team suites. Four adversarial test files target the consensus and ledger rules directly — twenty-one tests exercising attempted misbehaviour against the chain, the credit path, the delegation path and a second round of edge cases.

Fuzzing. Seventeen cargo-fuzz targets cover the attacker-controlled boundaries: transaction decoding, block decoding, wire-message decoding, mempool admission, storage recovery, checkpoint validation and codec, metadata parsing,

Layer-2 payload decoding, Layer-2 state application, the dispute arbitration engine, verified work certificate decoding, the machine-readable task readers, the deliverable codec, the worker decision flows, and two harnesses written for the authority and credit layers: one that drives grants, delegations, revocations and spends, and one that drives loans, collateral, repayments and defaults. The last two assert the properties those layers depend on — value conservation, that a spend never exceeds its grant, that revocation is absolute, that authority narrows, that debt only goes one way, that a default is not early, and that a replay is refused. A reported campaign of 9 352 161 executions produced no crashes and no sanitizer findings [17]; every registered target is now run in the continuous integration smoke loop, and every fuzz target has a deterministic counterpart in the ordinary test suite, so the properties the fuzzer checks are also checked on every CI run.

An independent implementation of the authority and credit rules. The rules of Section 7 are implemented a second time, in a different language, written from the specification rather than from the protocol's source. It derives the addresses, the agent identities, the grant hashes and the position identifiers itself, and emits a vector file of scenarios; the protocol's test suite replays those scenarios and asserts the same verdict, the same reason and the same final state. This is the same discipline the proof-of-work reference uses, applied to the part of the protocol where a rule — rather than a digest — is the thing that has to agree.

Attack simulation. Independent Python simulations reproduce the difficulty-adjustment behaviour under adversarial timing, the tail-emission schedule, and the puzzle digests, and a cross-check script verifies the consensus vectors against the Rust implementation.

Disclosure posture. The readiness record documents a ninety-day responsible-disclosure window, a severity model in which a chain-halting defect is critical and a key compromise is high, and a contact address — while noting honestly that the corresponding public key is not yet published, so the channel is not yet operational. We consider stating that plainly to be more useful than presenting the policy as if it were complete.

9.9 Known gaps

The following are known and are stated rather than concealed.

- No independent third-party security audit has been completed. The readiness record says so, and the recommendation stands: a third-party review should precede any deployment carrying real value [15].
- The disclosure contact exists but its public key is unpublished, so the channel is not yet operational.
- The specification and the code are not in perfect agreement everywhere; several drifted sections were corrected, but the specification remains a living document and the code is authoritative where they disagree in practice.
- The peer-to-peer layer has no discovery protocol and no peer authentication, and the Layer-2 dispute window admits a payment of an unchallenged result that happens to be incorrect.
- The network has not been subjected to load, eclipse or economic testing at mainnet scale.
- There is no validator yield yet. The decision is recorded — the reward comes from the Layer-2 fee pool, so the Layer-1 coinbase is untouched — but it is not implemented, because paying it means deciding which checkpoint a committee's work counts for, and the checkpoint a committee signs is not part of a block.
- There is no exchange: no second asset, no order book, no price oracle. The intent is that JNT's economic usage comes from JNT network usage.

10 Use by humans, machines, robots and agents

10.1 Identities and roles

The protocol defines three roles. A *human* is an ordinary wallet. A *machine* is a device spending under a policy written by its owner. A *service* is an identity that receives payments. The agent identifier is derived from the network identifier, the role and the owner, so the same device has different identifiers on different chains — a deliberate property, since an identity that is portable across chains is an identity that can be accumulated against on one chain and redeemed on another.

10.2 What an agent may do

An agent can hold value, receive value, spend value under a written grant, publish and revoke grants of its own, lend to another agent and be lent to, post and claim tasks, execute work, submit attested results, challenge results, and settle or be settled. What an agent cannot do is exceed the grant it was given, replay an authorisation, act without an owner signature somewhere in its ancestry, borrow without a lender's signature, or cause the protocol to create value. There is no second currency: the owner field is an ordinary address, so machine and human balances live in the same output set under the same issuance schedule.

10.3 Agent tooling

Worker SDK. The worker crate is a client, not a consensus component. It discovers open tasks, claims them, executes them, submits results, raises disputes and requests exits, and its decision functions are deterministic — no clock, no randomness — so the same inputs always produce the same plan. A dedicated security suite asserts that an actor starting with zero privilege cannot, through any path in the SDK, become an authorised worker, claimant, verifier, settler or payment recipient. A second module covers the authority and credit flows: publish a grant, spend under it, revoke it, open a loan, add collateral, repay, and settle a default — each one planned locally and then broadcast, like every other path in the SDK.

Model Context Protocol surface. The MCP crate is a keyless dispatcher over agent identity, payments, receipts and reputation, and over the authority and credit layer: an operation reads a published grant, a spend under a grant, or a loan, and returns the terms and the hash to sign. It is explicitly *not* a trust authority: it verifies using the same public functions every other verifier uses, and it cannot mint an authorisation, a payment or a receipt. It is explicitly *not* a key holder: no operation accepts, returns or stores a private key; where a signature is needed it returns the bytes and the signing hash, and the client signs. It holds an authorisation book so that anti-replay counters survive across calls, because a fresh book per call would silently reset every counter. Its transport is a small dependency-free HTTP client.

The command line. The node's own command line carries the same flows, because an operator should not have to write a program to grant an agent a budget: a deposit address and a Layer-2 summary, a grant publication, a spend, a revocation, and the credit operations — each of which checks the transaction against the state the chain will replay and refuses with the chain's own reason, rather than mining a block the replay would skip.

10.4 Concrete scenarios

A research pipeline buys a computation. A submitter posts a task with an escrow and a compute specification. A worker claims it, runs it, and submits a result hash with a signed attestation. If nobody challenges the result within the dispute window, the task settles correct and the worker is paid from the escrow; a verified work certificate is issued, and a relying party may insist on the quorum-backed kind if it needs independent proof.

A robot fleet operates under a budget. An operator publishes a grant bounding an agent per transaction, bounded per window, restricted to named recipients and named services, with an increasing nonce. The fleet spends within that envelope, and every node enforces it, not only the one that issued it. The operator revokes the grant, and every descendant grant dies with it.

A machine buys a service from another machine. A service identity publishes a receipt; the payer records the payment; the payment is settled only when bound to a Layer-1 transaction that actually pays the recipient. The receipt and the payment cannot be confused with one another, and a contradicting receipt for the same payment is refused.

A human buys the same service. The same paths apply. There is no separate human lane, no privileged account type, and no way for the protocol to distinguish a paying human from a paying robot other than by the identity they present.

11 Openness, governance and control of information

11.1 Licence and ownership

The code is released under a standard permissive Rust dual licence, `MIT OR Apache-2.0`, with both texts at the repository root [13]. Anyone may run it, study it, modify it, audit it, redistribute it, run a public node or a fork, use it as a dependency, write about it, and report vulnerabilities against it. Commercial use is permitted by the licence; the

Information	Where it lives	Who controls it
Ledger, transactions, task specifications	On chain, replicated	Nobody individually; rule-governed
Task results and attestations	On chain	Nobody individually
Exit records	Layer-2 state, committed by Layer-1 root	Nobody individually
Published grants, their counters	Layer-2 state, committed by the Layer-1 root	The owner, within the limits the grant states
Credit positions and their collateral	Layer-2 state, committed by the Layer-1 root	The lender and the borrower, by agreement
Wallet private keys	The holder's file, encrypted	The holder alone
Validator signing keys	The validator's encrypted file	The validator alone
Credit issuer key	The operator's configuration	The operator
Receipts, payment links, service-layer credit and escrow books	Node-local state file	The node operator, per node
Reputation	Derived view of chain events	Nobody; derived, not owned
Delegation lineage records	Node-local, signed by the owner	The owner, subject to revocation

Table 8: Information-control matrix. The distinction that matters is between information that is *committed to the chain* and therefore replicated and immutable, and information that is *node-local* and therefore belongs to one operator.

trademark document asks for a short written permission for >commercial use of the marks themselves}, which is a different question from commercial use of the code.

The project states that it is community-owned in the Bitcoin sense, and that no single person holds it. One inconsistency should be disclosed: the MIT licence text names an individual copyright holder, while the project's own documentation asserts community ownership. Reconciling those two statements is an open administrative task.

11.2 Trademark separation

The name, the ticker and the logos are community marks. Permitted without asking: running the software on any network, forking it, using the marks factually to refer to the project ("built on Jounit", in a compatibility list, in an article). Not permitted without asking: naming a product, company, chain or service "Jounit" or "JNT"; using the logos in one's own branding; publishing a fork under the name or implying official affiliation; selling something under the name; registering the marks as one's own.

The reasoning is worth stating because it is a general principle about permissively licensed infrastructure: a permissive code licence cannot protect a brand, and pretending otherwise fails. The code stays open; the name stays with the community; a fork takes a new name because two chains both displaying `JNT` is exactly the confusion that harms users.

11.3 Governance after launch

The project's stated principle is no post-launch governance: no foundation controls the protocol, no upgrade path is reserved for an insider, and the rules are the rules. Difficulty and puzzle parameters are already designed to be adjustable, but the supply ladder, the halving schedule and the base-fee split are stated to be immutable, so monetary policy cannot be changed by a software update. This is the Bitcoin-style commitment to a rule rather than to a committee.

11.4 Who holds and controls the information

Two properties follow from this table. First, no single party holds the information needed to reconstruct the global state of the system: the ledger is replicated, and node-local books are explicitly *not* an alternative source of truth for ownership or for spendable balance. A corrupt local book can cost an operator their own bookkeeping; it cannot mint a coin or move one. Second, the system is asymmetric on purpose: the operator's own books are private to them, and the shared financial record is public to all and owned by none. This is the same asymmetry that makes an open ledger useful for audit and a private wallet file useful for safety.

The node-local boundary is enforced by construction rather than by policy: the receipt, payment, service-layer credit and escrow books are not reachable from the consensus path, and the documented rule is that these books must never be

Command	Function
`init`	Create a chain directory, a miner wallet and the genesis block
`mine`	Mine blocks and extend the chain
`send`	Spend from a wallet address, locally or over the network
`status`	Chain tip, difficulty, supply, balance, Layer-2 pipeline
`verify`	Replay the chain from genesis and diff it against storage
`show-block`	Print one block
`balance`	List unspent outputs of an address
`wallet`	Create or list wallets
`policy`	Publish, revoke or list spending grants
`agent-spend`	Spend under a published grant
`credit`	Open, repay, default or list credit positions
`l2`	Layer-2 summary, and the deposit address
`validator`	Add or list checkpoint-signing keys
`export`	Write the canonical chain to JSON
`serve`	Run the node: relay, sync, API, exit crank, signing

Table 9: The node's command surface. Every command accepts `-help`, and every command accepts a network profile and a data directory.

treated as an alternative source of truth for global ownership or for a spendable balance. Coin movement happens only through Layer-1 outputs and Layer-2 account balances. The spending grants and the credit positions are on the other side of that line: they are consensus state, every node holds them, and they are covered by the state root.

12 Using Jounit

12.1 Building and running

The implementation requires Rust 1.75 or later (developed and tested against a much newer release) and Python 3.11 or later for the simulation suite.

```
git clone https://github.com/swamy18/Jounit.git
cd Jounit
cargo build --release
```

A chain directory, a miner wallet and the genesis block are created with `init`; blocks are produced with `mine`; the node reports its tip, difficulty, supply and balances with `status`; and `verify` replays the whole chain from genesis and diffs the result against stored state. `serve` runs the peer-to-peer node, mines by default so that matured exits settle, and optionally exposes the HTTP API.

```
export JNT_WALLET_PASSPHRASE=change-me
jnt-node init  --network testnet --data-dir ./chain
jnt-node mine  --network testnet --data-dir ./chain --blocks 25
jnt-node verify --network testnet --data-dir ./chain
jnt-node serve --network testnet --data-dir ./chain --listen 127.0.0.1:9833
```

For the HTTP API, a loopback bind needs no credentials; a non-loopback bind requires a bearer token, supplied through the environment rather than a flag, because flags are visible in the process table.

```
export JNT_API_TOKEN="..."
jnt-node serve --network testnet --data-dir ./chain --api-listen 0.0.0.0:8080
```

12.2 Command surface

12.3 Authority and credit, from the terminal

A grant is the owner's signed authority for a named spender; it is published on chain, so every node enforces it. A loan is the same idea for value.

Parameter	mainnet	testnet	mini
Block time	600 000 ms	1 000 ms	1 000 ms
Halving period	262 800	2 000	2 000
Start reward	76 JNT	76 JNT	76 JNT
Finality depth	100	20	20
Coinbase maturity	100	10	10
Genesis validators	none	4	4
Binding finality	height 100	height 20	off
Exits	height 1	height 1	height 1
Post-quantum requirement	off	off	off

Table 10: Network profiles. The mainnet profile deliberately ships with no genesis committee, so that every checkpoint fails closed until the stake-weighted committee derives from the bond ledger — the protocol never verifies against nothing.

```
# The deposit address: an ordinary Layer-1 payment to it credits a
# Layer-2 balance, which is what the grants and the loans spend from.
jnt-node l2 --deposit-address
jnt-node send --to <deposit-address> --amount 5

# Publish a grant: this key may spend up to 0.01 JNT per payment and
# 0.05 JNT per window, to this recipient only.
jnt-node policy --spender <spender> --per-tx 1000000 \
                --cumulative 5000000 --recipients <recipient>

# The agent spends under it.
jnt-node agent-spend --policy <policy-hash> --to <recipient> \
             --amount 0.002 --nonce 1

# The owner kills it; every descendant of the grant dies with it.
jnt-node policy --revoke --agent-id <agent-id>
jnt-node policy --list

# A loan: the lender signs, the principal moves, the collateral answers
# for the debt.
jnt-node credit --to <borrower> --amount 1 --collateral 100000000
jnt-node credit --repay --position <position-hash> --amount 1
jnt-node credit --list
```

Every one of these commands checks the transaction against the state the chain will replay and refuses with the chain's own reason, rather than mining a block the replay would skip.

12.4 Profiles

The three profiles differ only in numbers, and the `mini` profile is the same code path as production.

13 Limitations and future work

13.1 Limitations

The limitations of the present system are structural rather than incidental, and it is better to state them than to let a reader infer them.

The consensus layer is conventional. The fork choice is cumulative work on a linear chain. This buys predictability and a well-understood security model at the cost of throughput and of the latency properties that a DAG or a committee-based design would offer [24, 9]. Jounit's ambition is to be a settlement layer, and the design reflects that choice rather than transcending it.

Layer 2 inherits Layer 1's ordering and none of its latency. The replay model means Layer-2 state is always as fresh as the last Layer-1 block, and a Layer-2 payload that fails to apply is skipped rather than reverting the block. This is a deliberate trade of strictness for liveness.

The optimistic settlement model has a known cost. An unchallenged incorrect result is paid. The protocol narrows the consequence — a quorum-backed certificate is distinguishable from an unchallenged one — but does not remove it.

Peers are unauthenticated. The current transport has no node identity and no discovery protocol. A denial-of-service or eclipse attack against the peer-to-peer layer is mitigated by bounds and bans but not prevented by cryptography.

The Layer-2 dispute machinery depends on a committee. Committee membership is derived from bonded stake and capped at twenty-one members, so the adjudicator is small. That is a deliberate consequence of quorum certificates, and it concentrates the value of a successful compromise.

No field deployment. The chain has not been run at mainnet scale, under load, or against an adversarial peer-to-peer network. The verification record's benchmarks are single-machine measurements on modest hardware, and the fuzzing campaign is bounded rather than exhaustive.

13.2 Future work

In rough dependency order: an independent third-party security review and a published disclosure key; a peer-to-peer layer with node identity, peer authentication and a discovery mechanism; a larger committee with staggered rotation; a richer proof-of-work registry in which a memory-hard puzzle can be enabled by consensus rule rather than by configuration; a formal verification effort on the consensus transition function and the proof-of-work forward pass; and a long-horizon economic study of the issuance schedule under realistic fee activity, which would test the claim that net supply can fall.

14 Conclusion

Jounit is an attempt to build a settlement primitive that general-purpose cryptocurrencies do not provide: a way for a machine to be paid for work that a third party can verify without re-running it, and for a principal to delegate spending authority to such a machine without losing the ability to bound it.

The design is not exotic. It uses a linear UTXO chain with cumulative-work fork choice, a quorum certificate over a stake-weighted committee, a deterministic replay model for the second layer, and inclusion proofs to move value back across the layer boundary. What is unusual is the set of constraints applied throughout: no floating point in consensus, overflow refused rather than wrapped, a work-share band that prevents the interesting part of the proof of work from becoming the weak part, an escrow that names its own arbitrator and refuses oracle conditions it cannot evaluate, a reputation system that is structurally excluded from consensus, a receipt system that distinguishes "nobody objected" from "a quorum verified this", a presence-only rule for Layer-2 roots that converts a dishonest producer's power to steal into a power to stall, and a wallet file that is encrypted, atomically written and zeroised.

The newest part of the design is the least exotic and, we would argue, the most consequential: a machine's authority to spend, and a machine's ability to borrow, are stated as *rules the chain checks* rather than as records one node keeps. The grant binds an owner to a named spender and a named agent; the loan binds a lender to a borrower, a principal, a collateral, a rate and a deadline. Neither is novel as an idea. What is new is that they were node-local, and that closing that gap was mostly a matter of writing the rules down twice — once in a specification, once in a second implementation — and letting the two disagree until they did not.

The honest summary of the security posture is that the engineering is disciplined and the evidence is unusually well documented for a pre-launch project — a measured fuzzing campaign, red-team suites, independent re-implementations, a spec-versus-code reconciliation pass, and a disclosure of what was previously overstated — while an independent third-party review has not yet been performed. That is stated in the paper and in the project's own records because a protocol's credibility ultimately rests on the accuracy of its self-assessment.

The vision is a chain that does useful work with the energy it consumes, and that treats an autonomous agent as a first-class economic participant rather than as an account to be impersonated. Whether that vision survives contact with adversarial use is an open question, and the correct next step is the one that has not yet been taken.

Acknowledgments

The author thanks the contributors to the open-source cryptocurrency and agent-runtime ecosystems whose prior work this protocol builds on, and the reviewers of the specification whose disagreements improved it. The specification, the decision log and the verification record are the primary artefacts of this work and are openly available [14, 19, 16, 15, 27, 17].

References

[1] Richard Sutton. The bitter lesson. `https://incompleteideas.net/IncIdeas/BitterLesson.html`, 2019.

[2] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Iwa Lang, and Yushi Dai. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18, 2024.

[3] Qiusheng Wang et al. AutoGPT: An autonomous GPT-4 experiment. arXiv:2304.03442, 2023. `https://arxiv.org/abs/2304.03442`.

[4] John Yang et al. SWE-agent: Agent-computer interfaces enable automated software engineering. arXiv:2405.15793, 2024. `https://arxiv.org/abs/2405.15793`.

[5] Morgan Quan, Steve Lindsay, Jesse Doux, et al. Robot operating system (ROS): Maturity and recent trends. *Open Source Software*, 7(1):1–17, 2021.

[6] Zawy. Dark gravity: Overview. Blog post, 2019. `https://github.com/zawy12/darkgravity-pool`.

[7] CryptoNote. Cryptonote white paper. `https://cryptonote.org/whitepaper.pdf`, 2012.

[8] Cynthia Dwork and Moni Naor. Mining or not mining? that is the question. In *Proceedings of the 20th Annual Conference on Applied Cryptography and Information Security (ACNS)*, pages 178–195, 2012.

[9] Sarang Trivedi, Vipul Goyal, Vartika Vidyasagar, Sriram Rajamani, Alessandro Bonchi, and Jan Vondrak. Sok: Consensus mechanisms in blockchain. *ACM SIGACT News*, 48(2):305–321, 2017.

[10] Gavin Wood. Decentralized oracles: A primer. `https://gavwood.com/decentralized-oracles.pdf`, 2014.

[11] Satoshi Nakamoto. Bitcoin: A peer-to-peer electronic cash system. `https://bitcoin.org/bitcoin.pdf`, 2008. Self-published white paper.

[12] Joseph Bonneau, Alex Corrigan-Gibbs, Benjamin Kroll, Mary Maller, Andrew Milbourn, Ege Polat, Nithya Saxena, David Shepard, Pieter Wuille, and Adam Wright. Sok: Research perspectives and challenges for bitcoin and cryptocurrencies. *IEEE Symposium on Security and Privacy*, pages 474–491, 2018.

[13] Swamy Gadila. Jounit (JNT) protocol implementation. `https://github.com/swamy18/Jounit`, 2026. Rust workspace; MIT OR Apache-2.0.

[14] Swamy Gadila. Jounit protocol specification. `https://github.com/swamy18/Jounit/blob/main/SPEC.md`, 2026. Normative protocol document.

[15] Swamy Gadila. Jounit mainnet operational readiness record. `https://github.com/swamy18/Jounit/blob/main/docs/mainnet-readiness.md`, 2026.

[16] Swamy Gadila. Jounit verification record. `https://github.com/swamy18/Jounit/blob/main/docs/status.md`, 2026.

[17] Swamy Gadila. Jounit fuzzing campaign report. `https://github.com/swamy18/Jounit/blob/main/docs/fuzz.md`, 2026.

[18] Andreas M. Antonopoulos. *Mastering Bitcoin: Programming the Open Blockchain*. O'Reilly Media, Sebastopol, CA, 2 edition, 2017.

[19] Swamy Gadila. Jounit design decisions log. `https://github.com/swamy18/Jounit/blob/main/docs/decisions.md`, 2026. Decisions D1–D24, P1–P6 and M-A–M-E.

[20] National Institute of Standards and Technology. Secure hash standard (FIPS 180-4). `https://doi.org/10.6028/NIST.FIPS.180-4`, 2015.

[21] Peter W. Shor. Algorithms for quantum computation: Discrete logarithms and factoring. *Proceedings of the 35th Annual Symposium on Foundations of Computer Science*, pages 124–134, 1994.

[22] Lov K. Grover. A fast quantum mechanical algorithm for database search. In *Proceedings of the 28th Annual ACM Symposium on Theory of Computing*, pages 513–521, 1996.

[23] Swamy Gadila. Scoped security review of the HBM scratchpad proof-of-work. https://github.com/swamy18/Jounit/blob/main/docs/security-review-hbm-2026-10-12.md, 2026.

[24] Ittai Eyal and Emin Gün Sirer. On the security of bitcoin's proof-of-work. *ACM Transactions on Information and System Security*, 18(2):1–49, 2015.

[25] Stefanos Kiayias, Giorgos Leonardos, and Bracha Alman. Equitable mining beyond the proportionality theorem. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 203–217, 2017.

[26] Vitalik Buterin. EIP-1559: Fee market change for ETH 1.0 chain. Ethereum Improvement Proposals, 2020. https://eips.ethereum.org/EIPS/eip-1559.

[27] Swamy Gadila. Jounit monetary policy. https://github.com/swamy18/Jounit/blob/main/docs/monetary-policy.md, 2026.

[28] Dan Boneh, Craig Gentry, Ben Lynden, and Chris Peikert. Aggregate and verifiably aggregate signatures. *Advances in Cryptology — CRYPTO 2003*, 2729:416–432, 2003.

[29] H. Barnes, M. Bhargava, D. Boneh, E. Miller, Y. Salim, B. Warinschi, and B. Yuan. The BLS signature scheme. Cryptographic Research, IETF CFRG draft draft-irtf-cfrg-bls-signature-04, 2022. IETF CFRG.

[30] National Institute of Standards and Technology. Module-lattice-based digital signature standard (FIPS 204). https://doi.org/10.6028/NIST.FIPS.204, 2024.

[31] National Institute of Standards and Technology. Transition to post-quantum cryptography standards (NISTIR 8547). https://doi.org/10.6028/NIST.IR.8547.ipd, 2024.

[32] Yoav Nir and Alex Langley. The ChaCha20 and Poly1305 aead algorithms for internet protocols. RFC 8439, IETF, 2018. https://www.rfc-editor.org/rfc/rfc8439.

[33] Alex Biryukov, Daniel Daniel, Itai Dinur, and Wolfgang Keller. Argon2 password hashing function. RFC 9106, IETF, 2023. https://www.rfc-editor.org/rfc/rfc9106.

[34] Supranational. blst: A BLS12-381 implementation. https://github.com/supranational/blst, 2024. Reference pairing implementation.

[35] Dan Boneh, Ethan Drijvers, and Bruno Neven. Multisignatures secure in the plain public-key model, and a generalization of schnorr signatures. *Advances in Cryptology — CRYPTO 2018*, 10821:905–929, 2018.

A Key protocol constants

B Security invariants

The following invariants are asserted by the test suite, the fuzz targets, or both.

1. A block that does not satisfy the proof of work, or whose declared difficulty disagrees with its branch, is refused — and the difficulty check precedes the proof check.
2. No transaction may create value: the sum of inputs equals the sum of outputs plus the fee.
3. No block may mint beyond the issuance integral or the rate anchor.
4. The base fee is split exactly in half between burn and reserve, with the odd unit assigned to the burn.
5. A reorganisation that would cross the binding finalised point is refused.
6. A checkpoint is accepted only if it satisfies every clause of the seven-clause checklist, including quorum, signature and sidecar.
7. A layer-2 state transition preserves the conservation invariant.
8. An exit may be released at most once per (owner, nonce) pair, and only to the destination recorded in the proven exit record.
9. A settlement link binds one payment to at most one transaction, and the transaction must pay the recipient.
10. A credit attestation may mint at most once, ever, including across a restart.
11. A spending authorisation cannot be replayed: the nonce must strictly increase.
12. A destroyed state file is a refusal to start, never an empty book.
13. A lying layer-2 root is accepted at layer 1 but arms no exit.
14. The API never signs, never holds a private key, and never reaches the consensus submission path.

Quantity	Where	Value
Block header size	block	177 bytes
Transaction output size	block	28 bytes
Address size	hash	20 bytes
Agent identity size	agent	20 bytes
Grant hash size	agent	32 bytes
Gas per transaction	transaction	16 000
Gas per input	transaction	1 000
Gas per output	transaction	250
Maximum transactions per block	parameters	5 000
Maximum block size	parameters	4 000 000 bytes
Difficulty window	parameters	60 blocks
Maximum difficulty change	parameters	10 %
Median look-back	difficulty	11 blocks
Maximum attempts per block	chain	200 000 000
Quorum threshold	validators	$\lceil 2n/3 \rceil$, basis points 6 667
Maximum committee size	validators	21
Epoch length	validators	100 blocks
Unbonding period	bond	1 008 blocks
Slashing vesting period	slashing	52 560 blocks
Escrow expiry	escrow	1 008 blocks
Layer-2 deposit minimum	parameters	1 000 000 subunits
Layer-2 flat fee minimum	parameters	1 000 subunits
Exit fee minimum	exit	1 000 subunits
Whistleblower bounty	task contract	25 %
Layer-2 transaction codes	layer 2	`0x10`–`0x1C`
Grant / credit leaf tags	state root	`0x05` / `0x06`
Delegation depth cap	delegation	8
Grant publication fee	parameters	2 000
Grant revocation fee	parameters	2 000
Agent spend fee	parameters	2 000
Credit open / collateral / repay / default fee	parameters	2 000 each
Maximum credit rate per window	parameters	1 000 bps
Maximum frame payload	network	4 100 000 bytes
Maximum frame payload	network	4 100 000 bytes
Maximum peers (total / inbound)	network	125 / 32
Maximum API connections / concurrency	API	128 / 64
API request timeout / header timeout	API	10 s / 30 s
Token bucket rate / capacity	API	60 per second / 120
Maximum state container size	store	64 MiB

Table 11: Selected constants. Every value is a named constant in the source rather than a literal at a use site, and every profile derives from the same parameters module [14].